

Agentic Publication Protocol: An Attempt to Modernize Scientific Publication

Sirui Lu^{1,2} and Xiao-Liang Qi^{3,*}

¹Max-Planck-Institut für Quantenoptik, Garching, Germany

²Munich Center for Quantum Science and Technology (MCQST),
Schellingstrasse 4, D-80799 München, Germany

³Leinweber Institute for Theoretical Physics, Stanford University, Stanford, CA, USA
*xlqi@stanford.edu

Abstract

Scientific publication is still organized primarily around static manuscripts, even though much of scientific progress depends on tacit know-how: how to run code, reproduce figures, interpret edge cases, choose useful follow-up directions, and avoid failed paths. Large language model agents create an opportunity to publish not only knowledge, but also operational know-how in a form that future readers and researchers can directly use. This paper outlines the Agentic Publication Protocol (APP), a lightweight repository format for packaging a paper together with code, data, environment information, reproducibility instructions, and an agent-facing instruction file. APP treats a version-controlled repository as the publication object and uses `AGENTS.md` and optional skills to define a paper agent that can explain the work, reproduce key results when possible, and support follow-up research. We describe the design principles and details of the protocol, as well as the agent skills useful for publishing papers under the protocol. We also describe development tools for evaluating and improving the protocol and associated agent skills. Finally, we provide a broader discussion of the future of scientific research in the agent era.

1 Introduction

Scientific research is a long-term, collaborative process involving the entire research community. In the current research paradigm, scientific papers are the primary medium for sharing discoveries. In addition, they also serve two other important roles. Posting and publishing a paper is the main way to establish priority and credit. The impact of papers is one of the main metrics for evaluating and rewarding researchers. Peer review, journal publication, and citation are all built around the paper as the central scholarly object.

However, the static paper format has long been criticized across different domains for a number of limitations. More specifically, it has the following problems, which cause friction and burden in the research process:

1. **Limited, static representation of information.** The paper format is rigid and static. It can only target a particular group of readers. For example, a paper published in a specialized journal may be difficult to understand for readers outside the subfield, while a paper published in a general journal may emphasize a general idea that is attractive to a broader audience but correspondingly sacrifice technical precision and detailed information.
2. **Failure to update.** A paper, once published or posted, is difficult to update. Improvements or corrections that are useful for the reader can only be made in a new version when they are significant enough. Suggestions for improvement from readers can only be manually incorporated by the authors. In the current system, the authors often lack sufficient incentives to update the paper. Especially after a paper is published in a journal, there is no way to make such updates other than publishing an erratum or a new follow-up paper. In comparison, an open-source project on GitHub can be updated easily by the authors and improved by contributions from the entire community.

3. **Lack of reproducibility.** Many papers do not provide sufficient information to reproduce the key results, such as code, data, environment details, or instructions [1, 2]. This makes it difficult for readers to verify the claims, build on the work, or learn from the methods. Even under journal data-sharing policies, one study of *Science* papers could obtain artifacts for only 44% of them and reproduce the main findings for only 26% [3]. Even for papers with code and data, the reproduction process can be quite nontrivial, requiring the reader to spend a lot of time and effort to understand the code, set up the environment, and run the scripts. This creates a high barrier for follow-up research and slows down scientific progress.
4. **Missing know-how.** A paper may describe the final conclusions of a project, but it often omits the operational knowledge required to build on the work: which scripts actually generate which figures, which numerical settings matter, which derivations are fragile, which data files are authoritative, and which attempted directions failed. Much of this is tacit knowledge in Polanyi’s sense: we know more than we can tell [4]. Such know-how is often transferred only through direct communication with the authors or through trial and error by readers.

With the rapid development of large language models (LLMs) in recent years, LLM agents play an increasingly important role in scientific research [5–8]. Agents can already plan and execute experiments or simulations [9–11], contribute to theoretical and mathematical discoveries [12–20], and support end-to-end research workflows from literature review and implementation to experiments and report writing [21–25]. In a recent article by one of us [6], we call this process the agentification of scientific research, which we expect to substantially change research practice in many fields. This significant change is a double-edged sword. On one hand, it brings more challenges to the outdated system of scientific papers. The accelerated research process creates a more urgent need to renovate the current paper format and the peer-review and publication procedures. On the other hand, it also creates a new opportunity to modernize scientific publication. With the help of LLM agents, we can make new attempts to overcome the problems of the publication system discussed above. This is the motivation for us to propose the **Agentic Publication Protocol (APP)**¹.

APP is a proposal for publishing scientific work in a new format that is more dynamic and reproducible, and that preserves not only the knowledge but also the know-how of scientific work. By following the APP protocol, information about a paper is organized in a more modular way, and additional information is provided to enhance reproducibility of the key results in the paper. By including detailed instructions, the paper becomes more ready to be read and reproduced with the help of LLM agents. The goal is that an agent is delivered to the reader together with the paper contents, so that the reader can interact with a representative of the author to understand and reproduce the paper, and can therefore focus on more creative tasks such as carrying out a related research project. Having an agent represent each paper may also enable new forms of scientific collaboration, such as new ideas and follow-up directions proposed through interactions between paper agents.

The APP git repository² contains two parts: the protocol itself, defined by `PROTOCOL.md`, and a set of agent skills that assist the authors in preparing an APP publication. The skill `publish-paper`, which uses several other skills in each step, guides the author through the procedure of organizing a paper into an APP-compliant format and publishing it as a release of a GitHub repository. However, using the skills we provide is optional. The protocol is well-defined independently of the skills, and papers compliant with the protocol can also be built in any other way, either manually or with the assistance of other agents.

Agentification of scientific publication is a very new development, and we would like to invite the research community to join us in improving both the protocol itself and the skills that implement the protocol. Researchers in different fields likely have different emphases or domain know-how that are essential for a paper agent. Instead of a universal `publish-paper` skill, domain-specific agent skills may be developed. To provide a starting point for further development of APP, we share a **development repository** that contains agent skills and tools for evaluating and improving the protocol and associated skills.

The remainder of the paper is organized as follows. In Section 2, we give a detailed overview of the protocol, including the design principles, the general structure of an APP publication, the publication workflow, and

¹<https://github.com/LionSR/AgenticPublicationProtocol>

²We use standard version-control terminology throughout the paper. A (*git*) *repository* or *repo* is a folder whose entire history of changes is tracked; a *commit* is one recorded snapshot of that folder; a *tag/release* marks a specific commit as a named, citable version; and a *manifest* is a small machine-readable file that records exactly which snapshot is being claimed as the publication.

the corresponding skills. In Section 3, we discuss the development tools for improving the APP protocol and evaluate the resulting paper agents on example papers. In Section 4, we discuss other works related to agentic publication. Finally, in Section 5, we conclude and further discuss the future of scientific research in the agent era.

2 Protocol Overview

2.1 Design Principle

To address the problems discussed in Section 1, we want the APP protocol to satisfy the following principles:

1. **Organized information.** Information in the publication should be organized in a modular way to minimize ambiguity and maximize usability. The repository structure should be modular enough that both humans and agents can locate the relevant material quickly. The distinction between “ground truth” of the work and optional auxiliary material should be clear.
2. **Enhanced reproducibility.** The publication should make key results as ready as possible for reproduction by a reader or agent. Whenever possible, the publication should include the commands, scripts, data, and environment needed to reproduce essential results in the paper, including figures, tables, and analytic derivations.
3. **Version control and timestamping.** The publication should be a version-controlled object with a unique identifier. A git commit, release tag, and manifest can establish the exact file tree being claimed as the publication, reducing ambiguity about priority and provenance.
4. **Know-how sharing.** The publication should include instructions that teach an agent how to represent and use the work. In addition to representing the key results of the work itself, the agent can also represent the authors’ broader know-how if the authors decide to share it in the form of agent skills.

2.2 More Details on the Protocol

The APP protocol is defined by [PROTOCOL.md](#). It specifies the required repository structure, metadata, validation expectations, and publication object. The protocol is designed to be lightweight and flexible, allowing authors to choose how much information to include and how to organize it, as long as the required files and structure are present. In the APP protocol, a publication is defined as a public git repository together with a specific release. The repository is not only a place to store the paper source. It is the publication object itself. The protocol specifies what files should be included, which files are authoritative, how an agent should read the repository, and how a reader can verify which snapshot is being claimed as the published work. It does not prescribe how the repository has to be created. The skills used to prepare, check, or release an APP publication are useful tools, but they are separate from the protocol itself.

A complete APP publication has three layers. The first layer is the human publication layer, which includes the manuscript, README, license, code, data, and environment information. The second layer is the agent-facing layer, centered on [AGENTS.md](#), which tells an agent how to represent the paper faithfully and how to navigate the repository. The third layer is the release and verification layer, which binds the repository to a tag, commit, manifest, and author approval. These layers together make the publication readable by humans, usable by agents, and identifiable as a stable scholarly object.

A typical APP repository has the following structure (see Fig. 1):

Canonical structure of an APP publication repo

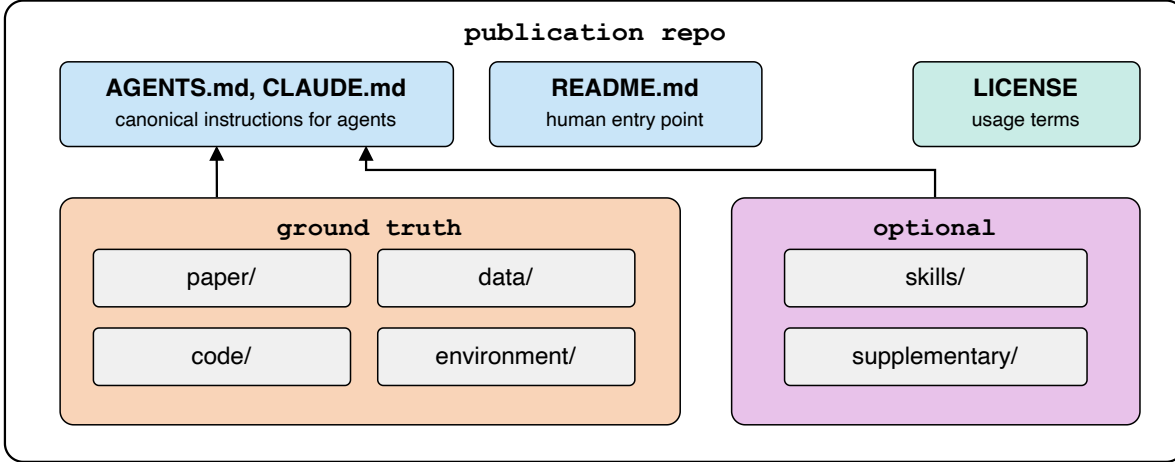


Figure 1: Repository structure of an APP publication. The repository contains the paper, human-facing documentation, agent-facing instructions, and the optional code, data, environment, supplementary material, and skills that are needed to make the paper usable as an agent-readable publication object.

Path	Purpose
AGENTS.md	Primary instructions for the paper agent
CLAUDE.md	Optional pointer for Claude Code compatibility
README.md	Human-facing overview and usage instructions
LICENSE	Reuse terms
paper/	Canonical manuscript and paper figures
code/	Source code and reproduction scripts
data/	Local data or data access documentation
environment/	Dependency and runtime setup instructions
supplementary/	Author notes, derivations, transcripts, or context
skills/	Optional paper-specific agent skills

AGENTS.md, README.md, LICENSE, and **paper/** are required for every APP publication. Other directories are included when they are relevant to the work. For example, a theory paper with no code or data may omit **code/** and **data/**, while a numerical or experimental paper should include the scripts, data description, and environment information needed for reproduction.

The **paper/** directory contains the canonical paper. This may include the manuscript source, compiled PDF, figures, and bibliography. Exactly one document should be identified as the canonical paper, and its format and path should be declared in AGENTS.md. The **code/** directory contains source code and scripts that belong to the publication. When figures or tables are generated computationally, the protocol requires a figure/table reproduction area, usually **code/figure-reproduction/**, with a README.md that maps each paper figure or table to the corresponding script, inputs, generated output, status, and any blocker. The **data/** directory contains local datasets when they are small enough to ship with the repository. If data are external, **data/README.md** should describe where they come from, how to obtain them, and which scripts or figures use them.

The **environment/** directory records how to recreate the computational environment. It may contain files such as **requirements.txt**, **environment.yml**, lockfiles, a **Dockerfile**, or other dependency specifications. If code or nontrivial build tools are needed, the protocol requires **environment/README.md** to state the tested platform, setup commands, and the command prefix that an agent should use to run the code. Installed environments and package caches should not be committed; they should be recreated from the committed dependency files.

The central agent-facing file is **AGENTS.md**. This file contains machine-readable frontmatter and a human-readable body. The frontmatter records metadata such as the protocol name, protocol version, paper title, authors, canonical paper format, publication version, domain, and optional external references. The body should include the identity of the paper agent, a paper summary, key results, repository structure, concrete things the agent can do, computational requirements, and citation information. In this way, **AGENTS.md** is not a replacement for the paper. It is an instruction file that teaches an agent how to use the paper, code, data, and environment as the ground truth.

The protocol also distinguishes several levels of status. A repository with an **AGENTS.md** file is agent-readable. A repository whose **AGENTS.md** frontmatter declares the APP protocol is an APP-structured candidate. A verified APP publication must additionally correspond to a tagged release with an **APP_PUBLICATION.json** manifest. The manifest records the repository URL, tag, commit, tree hash, validation result, and human approval. This makes the publication a specific versioned object rather than only a mutable folder of files.

2.3 Publication Workflow and Skills

To make it easier for authors to prepare an APP publication, we provide a set of agent skills in **skills/** that guide the author through the workflow. The protocol and the publication skills are conceptually separated. The protocol specifies the required repository structure, metadata, validation expectations, and publication object. Skills provide one convenient way to prepare such a publication, but authors should not be required to use a particular tool. Authors can follow the protocol manually, or use other tools or agents they prefer.

In the following, we provide an overview of the publication workflow defined in the **publish-paper** skill, which uses several other skills in each step. The workflow is designed to be flexible. Authors can choose to run the stages individually or run the complete workflow through the **publish-paper** metaskill. The overall workflow is illustrated in Fig. 2(a), and more detailed steps are illustrated in Fig. 2(b). The workflow starts from an original working repository, which may be a private or public repository that contains the source material of the paper. It ends with a validated APP publication release: a tagged release of the repository that corresponds to a specific commit and tree hash and has passed the validation checks.

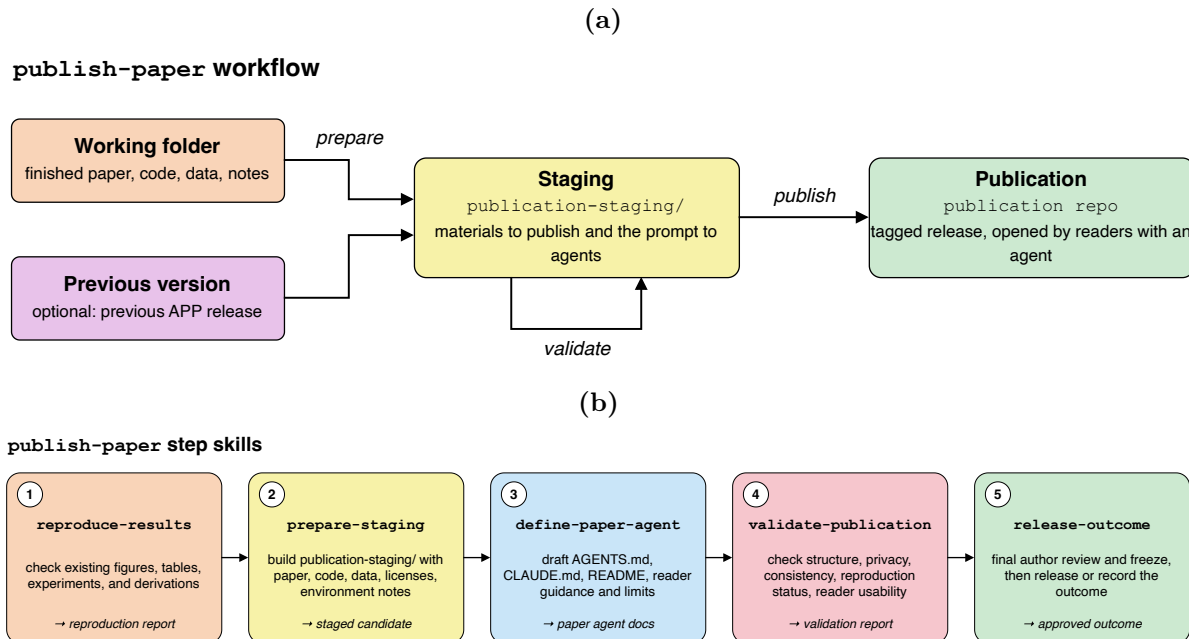


Figure 2: Publication workflow supported by APP skills. (a) Overall workflow from a working paper repository to a validated APP release. (b) The **publish-paper** metaskill decomposes this process into step-specific skills that reproduce results, prepare staging, define the paper agent, validate the publication, and release the outcome.

The **publish-paper** workflow can be organized into five stages.

1. **reproduce-results.**

The first stage checks existing paper results before files are reorganized. The goal is to make the key figures, tables, experiments, and analytic results as reproducible as possible. Wrapper scripts should be written when useful, and generated figures should be visually compared with the manuscript figures. The outcome is a reproduction report recording which results are reproduced, blocked, manual-only, or inconsistent.

2. **prepare-staging.**

The second stage organizes the source material into a **publication-staging/** folder following the APP layout. Files are copied or summarized into their public locations, dependencies are documented, and private or irrelevant working files are excluded.

3. **define-paper-agent.**

The third stage creates the paper-agent documentation, especially **AGENTS.md**. This file should describe the paper’s contribution, ground truth hierarchy, repository structure, setup instructions, reproduction map, limitations, and safe follow-up directions. It is helpful to keep **AGENTS.md** concise by pointing to canonical files such as **README.md**, figure-reproduction notes, data documentation, and environment instructions rather than repeating their details. Optional paper-specific skills can also be added or referred to.

4. **validate-publication.**

The fourth stage reviews the staged publication for structural completeness, path consistency, privacy issues, factual consistency, environment setup, and reader-agent usability. Validation should not act as peer review or judge scientific novelty.

5. **release-outcome.**

The final stage records author approval and creates the publication object, such as a tagged release with a manifest. This release binds the repository URL, commit, tree hash, validation result, and author approval into a stable identifier.

In addition to the **publish-paper** workflow and its step skills, APP includes auxiliary skills for more targeted interactions with agent-readable papers. The skill **load-paper** is a reader-side utility. It loads a published paper repository, a local **publication-staging/** candidate, a non-APP repository, or an arXiv paper into the current project so that an agent can inspect the paper, classify its APP status, locate its documentation and code, and help a reader consult or build on the work. When the input is a local staging folder, the skill treats that folder as a candidate publication root, but does not claim that it is a verified APP release.

The skill **extract-chat-context** helps authors preserve useful research know-how from prior agent sessions. It extracts Claude Code or Codex conversation history, summarizes the important decisions, debugging insights, methodological choices, and dead ends, and prepares publication-safe supplementary material such as a **know-how.md** file. This skill is useful when interaction with agents during research is itself valuable context for future readers. When the sessions contain recurring procedures that the research actually validated, the skill can also propose packaging them as agent skills, either bundled with the publication or referenced as external skills. Because chat histories can contain private notes, credentials, or unapproved claims, the skill includes author review and privacy screening before any extracted context is copied into **publication-staging/**.

The skill **create-paper-page** supports distribution after a paper has been prepared. It creates a lightweight GitHub Pages project page for a paper, including the title, authors, abstract, highlights, selected figures, links to the paper, code, data, citation information, and an indication that the paper has an associated agent-readable repository. The page lives under **supplementary/** as supplementary material and is deployed to GitHub Pages by a workflow, so it is versioned with the tagged release while the tree keeps the protocol layout, and its links route readers to the publication repository rather than to raw repository files. This page is not part of APP compliance, but it provides a familiar public entry point for human readers while linking them to the richer APP publication package.

These auxiliary skills are not required by the protocol. Rather, they illustrate how APP repositories can become active research objects. Once the paper, code, data, environment, and agent instructions are

organized in a standard way, different agents can reuse the same publication package for reading, reproduction, review, presentation, and follow-up work.

3 Agent-Assisted Development and Evaluation

3.1 Agentic Development Tools

We do not intend to present APP as an already mature package of protocols and tools. Instead, our goal is to jump-start a process through which the entire research community can develop this new protocol together. To develop and improve APP, we defined several agent skills that test the performance of the protocol. In addition to sharing APP, we would also like to share a [development repository](#) with these skills, since it may be useful for researchers who are interested in contributing to the improvement of this protocol or of the development tools themselves. The goal of these agentic tools is to make the future development of APP as smooth and automatic as possible. In this section, we provide an overview of the development repository.

The published development repository is organized using the APP layout itself. The main development materials have the following structure:

Path	Role in development
<code>paper/</code>	Manuscript and figures describing APP
<code>code/protocol_repo/</code>	Public APP protocol repository, included as a submodule
<code>code/scripts/</code>	Dataset, plotting, bibliography, and maintenance scripts
<code>data/example-papers/</code>	Source paper projects used as realistic author workspaces
<code>data/compare-app-benchmark/</code>	Compact public <code>compare-app</code> summary data
<code>skills/</code>	Development skills and automated test runners
<code>supplementary/</code>	Supplementary materials, including the project video
<code>working/</code>	Local generated transcripts, logs, reports, and design notes, not committed in the first release

`data/example-papers/` is the public location for example source materials extracted from already published papers. We use these examples to simulate the materials (paper, code, data, notes, etc.) that authors may have at the start of the publication workflow. In the first published release, this directory is intentionally empty. Example papers can be added here as starting points for the development workflow. Some example papers are shared through our [Hugging Face dataset](#). A compact summary of the evaluation results discussed below, used in Table 1 and Fig. 4, is included under `data/compare-app-benchmark/`, while the larger transcript-level benchmark records are also hosted in the [Hugging Face dataset](#).

`skills/` contains the development skills used to test the protocol. The skill `simulate-publication` creates a simulated author and a publishing agent, each of which is a fresh Codex session, and runs the publication workflow in a development sandbox. The conversation history is recorded and analyzed by an evaluation agent to generate an evaluation report that points out the pros and cons of the publication procedure. The output of the publishing process is a subfolder, `data/example-papers/<example_name>/publication-staging/`, in the example source folder. The chat history and evaluation report are saved under the local `working/` tree when the skills are run locally, which serves as a regression test for the protocol. This generated `working/` directory is not part of the first published release. After a protocol or skill change, the same source project can be tested again to see whether the workflow has become more reliable and whether previous problems have been fixed. Before each new test, the staging folder is cleared, so the agent has to prepare the staging folder from the source materials again.

After a staged publication has been created, `test-paper-agent` tests whether the resulting APP paper agent is useful to a realistic reader. It launches a simulated graduate-student reader and a paper agent inside the staged publication folder, records their conversation, and asks an evaluator agent to judge whether the paper agent answered from the APP materials, supported understanding of the paper, and gave actionable guidance for reproduction or follow-up work.

The skill `compare-app` provides a controlled comparison between an APP paper agent and a general repository-aware agent. It does not require a prior `simulate-publication` or `test-paper-agent` run; it only requires an original source workspace and a separate APP staging folder containing the paper and agent instructions. The staging folder may have been produced by `simulate-publication`, by a manual APP preparation workflow, or by any other compatible process. `compare-app` copies the paper into a paper-only

reader context, generates one neutral question script from that paper context, asks the same questions to the APP paper agent in the staging folder and to the general agent in the source repository, anonymizes the two transcripts, and evaluates them without revealing which agent used the APP package. The evaluator scores four criteria: accuracy, informativeness, grounding in concrete source artifacts, and honesty about evidence and reproduction limits. Results of this evaluation on a set of example papers will be reported in Sec. 3, Table 1.

For these three skills, an orchestrator agent can launch multiple parallel sessions to test each example, which is helpful for speeding up the development process.

The final development skill, **find-reproducible-papers**, helps expand the example set itself. Its goal is to find papers that are useful for evaluating and improving the APP protocol. It searches for candidate arXiv papers by topic and date, then ranks them with documented signals of reproducibility. These signals include official code links, open-source licenses, repository activity, data availability statements, explicit discussion of code or reproducibility, checkable symbolic or analytic material, reproducible figures or tables, available linked artifacts such as models or datasets, and whether the work appears to require heavy compute. The shortlisted papers are then assessed more deeply by loading the paper, inspecting the code and data, and attempting to reproduce one key result.

3.2 Evaluation Results

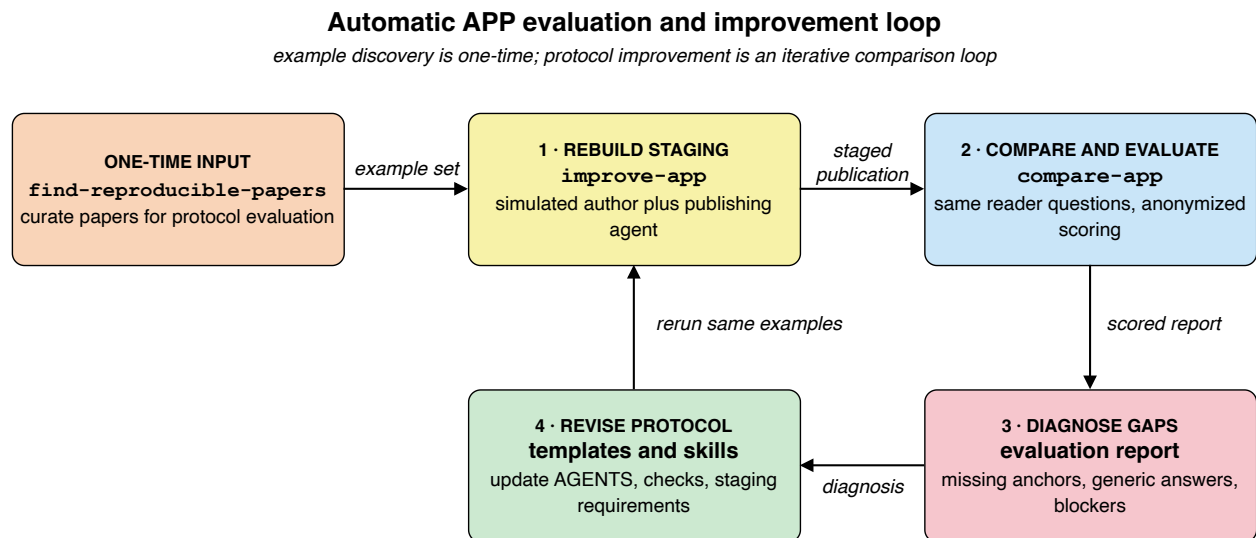


Figure 3: Automatic APP evaluation and improvement workflow. Development skills first help find suitable examples and rebuild APP staging folders. The controlled **compare-app** benchmark then probes the APP paper agent and a general repository-aware agent with the same reader questions. Evaluator reports are converted into protocol and template revisions, after which the same examples are rebuilt and tested again.

Using the development tools described in the previous subsection, we implemented an automatic evaluation and improvement workflow, illustrated in Fig. 3. The workflow iteratively evaluates and revises the APP protocol and its agent-facing templates. More specifically, the iteration proceeds as follows:

1. **Gather example papers.** A set of already published papers is curated as source workspaces under `data/example-papers/`.
2. **Simulate the APP publication procedure.** `simulate-publication` runs the publication workflow in a development sandbox. A simulated author and a publishing agent interact to prepare the APP staging folder `data/example-papers/<example_name>/publication-staging/`. The full transcript and evaluator report are saved as development evidence.

3. **Evaluate the agent performance.** Using the `compare-app` skill, we evaluate the paper agent built by `simulate-publication` against a general agent with access to the original source material. The two transcripts (with the same questions asked by the simulated reader) are anonymized and judged by a neutral evaluator according to the four criteria of accuracy, informativeness, grounding, and honesty. The evaluator reports and transcripts are converted into a short diagnosis of failure modes. This diagnosis is the basis for the next step of protocol revision.
4. **Revise the protocol documents.** The protocol templates, staging requirements, or development skills are revised according to this diagnosis. During this procedure, we direct the agent to minimize changes to protocol documents and mainly change the template `template/AGENTS.md` and the `define-paper-agent` skill.
5. **Iterate until satisfactory performance.** After the revision, we return to step 2 to rebuild the publication repositories and re-evaluate them on the same example papers and question sets. The general-agent transcripts can be reused. This isolates the effect of the protocol change and avoids attributing progress to different questions or a different baseline run. The loop continues until the paper agent’s performance clearly surpasses the general-agent baseline on most examples.

Paper	APP agent					General agent					Result
	Acc.	Info.	Grnd.	Hon.	Avg.	Acc.	Info.	Grnd.	Hon.	Avg.	
arXiv:2006.12469 [26]	9	9	10	10	9.50	9	9	9	8	8.75	APP
arXiv:2412.03356 [27]	9	10	8	9	9.00	9	8	9	8	8.50	APP
arXiv:2005.12702 [28]	9	9	9	9	9.00	8	8	8	8	8.00	APP
arXiv:1703.10587 [29]	9	9	9	9	9.00	9	8	8	8	8.25	APP
arXiv:2411.03110 [30]	9	10	10	9	9.50	9	9	9	9	9.00	APP
arXiv:2203.09758 [31]	9	10	10	10	9.75	9	8	9	9	8.75	APP
arXiv:2605.13271 [32]	9	9	10	10	9.50	9	9	9	9	9.00	APP
arXiv:2306.12711 [33]	9	9	10	10	9.50	8	8	8	8	8.00	APP
arXiv:2310.02075 [34]	9	9	8	9	8.75	9	8	8	9	8.50	APP
arXiv:2007.06989 [35]	9	9	10	10	9.50	9	9	8	9	8.75	APP
arXiv:2012.01459 [36]	8	9	9	9	8.75	8	8	8	8	8.00	APP

Table 1: Public-paper subset of the fixed-harness `compare-app` evaluation. Scores are on a 10-point scale for accuracy, informativeness, grounding, and honesty; the average column is the arithmetic mean of these four aspect scores. The neutral evaluator is Codex (`gpt-5.5`, reasoning effort `xhigh`).

Using `find-reproducible-papers`, we built an evaluation set of 11 arXiv papers in quantum physics [26–36]. Using 6 of these papers [26, 28, 31–33, 35] and a few other examples based on our own ongoing research work, we ran the automatic improvement loop. Initially, the paper agent had several concrete weaknesses. Early comparisons showed that the paper agent could be too distracted by protocol details and could answer from generic summaries rather than concrete source files or reproduction artifacts. After several iterations, the `define-paper-agent` skill was improved and can generate a more concise `AGENTS.md`. The performance of the improved paper agent versus the general agent is shown in Table 1 and Fig. 4. On this data set, the APP paper agent won on all 11 papers. The mean overall score was 9.25 for the APP paper agent and 8.50 for the general agent. We also observe that performance on the “test set” papers is comparable to performance on the “training set” papers. The largest advantage of the paper agent compared with the general agent is in grounding and honesty, which is aligned with the goal of APP.

All simulated sessions reported here were run with the OpenAI Codex CLI (versions 0.136–0.137, default model `gpt-5.5`, reasoning effort `xhigh`) in its non-interactive `codex exec` mode. The protocol itself is agent-agnostic, and the evaluation can be repeated with any agent CLI that supports scripted sessions.

Obviously, the advantage of the paper agent (built by our simulated authors) is only by a small margin, and the agent-carried evaluation may also have its own bias. The main purpose of our discussion in this section is to provide a starting point for agent-assisted development and evaluation of the APP protocol and

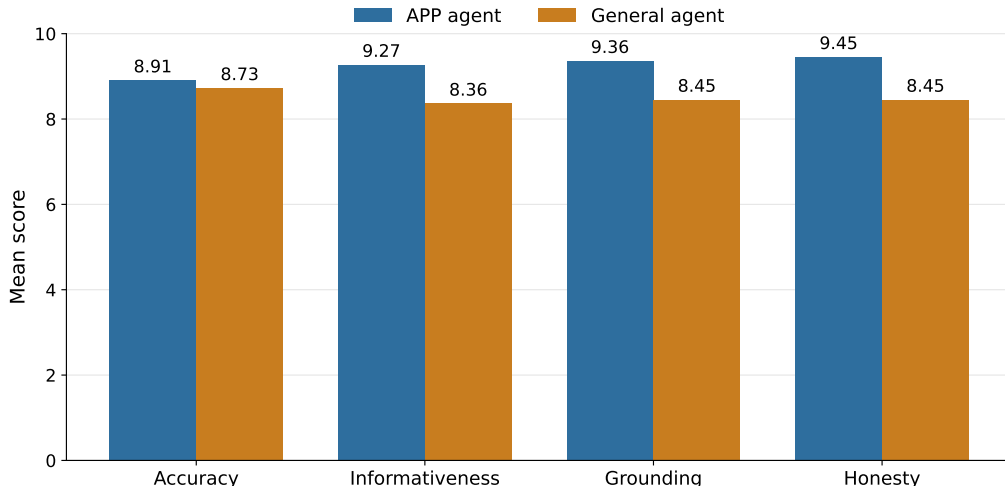


Figure 4: Average **compare-app** aspect scores over the 11 public papers.

corresponding skills. For example, a researcher interested in publishing a paper in APP can build the paper agent by herself, or with the **publish-paper** workflow, and evaluate it using the development skills provided here. We expect researchers in different fields to develop different approaches to building APP publications that can explore the further potential of this new way of sharing scientific discoveries.

4 Related Works

The limitations of the static paper discussed in Section 1 have motivated a long line of work that predates LLM agents. Research Objects proposed aggregating the paper together with data, methods, and provenance into a single shareable scholarly unit [37]; the FAIR principles made machine-actionability a community standard for research data [38]; and tools such as Binder build interactive computational environments directly from the configuration files of a public repository [39]. Many venues and platforms have also institutionalized reproducibility checklists [40], artifact evaluation [41], and paper-code links, the latter popularized by Papers with Code and now hosted by Hugging Face [42]. APP inherits the goals of this infrastructure but changes the consumer: the artifacts are packaged with explicit instructions so that an agent can operate the publication on the reader’s behalf, in a decentralized form that relies only on git and public hosting conventions.

A closely related proposal is the Agent-Native Research Artifact (ARA) [43]. ARA and APP share the view that a narrative paper plus an unstructured code repository is insufficient for agents to faithfully understand, reproduce, and extend research. ARA responds by proposing a typed, layered research object whose narrative paper is a compiled view of richer machine-readable content, including scientific logic, executable components, exploration traces, and claim-level evidence. APP takes a lighter operating point: it is a publication packaging and release protocol, not a universal ontology for research content. It keeps the manuscript authoritative, uses a familiar repository layout, and adds **AGENTS.md** plus a release manifest so that the publication is agent-readable, auditable, and verifiable. The two approaches are therefore complementary: ARA-like claim-evidence maps and research traces could serve as optional deep-knowledge layers inside an APP release, while APP provides the stable, field-neutral public release object.

The vision of the paper agent as a new interface to a publication [6] is shared across an emerging line of work. Paper2Agent [44] realizes one version of this vision by converting an existing paper and its codebase into a Model Context Protocol (MCP) server [45] so that a reader’s agent can invoke the paper’s methods as tools. Paper2Agent is a third-party method for turning existing papers into agentic artifacts, whereas APP is an author-side protocol for releasing such artifacts at publication time. The two are compatible: a Paper2Agent-style tool can consume an APP repository as a more reliable starting point. In a complementary direction, Paper2Code reconstructs an executable code repository from the paper text alone [46]. APP reduces the need for such reconstruction by shipping the authoritative code, environment, and reproduction

map with the publication.

More broadly, recent work on LLMs in science shows that agents are beginning to participate across the whole research cycle, from generating ideas and running experiments to writing papers and assisting peer review [21, 47]. LLM-generated manuscript feedback can overlap with human expert reviews [48], and AI-assisted reviewing is already visible in major conferences [49]. At the same time, empirical studies find growing LLM use in scientific writing and changes in manuscript production, citation behavior, and literature discovery [50, 51], while commentary emphasizes both the promise of AI-assisted communication and risks to depth, accountability, and trust [52]. These developments expose practical risks such as hallucinated claims or references, fragile implementation details, prompt injection, sycophantic review, and unclear human–AI contribution boundaries. Major venues and journals have responded with policies requiring disclosure of LLM use and keeping humans responsible for published content [53–57]. Across different stages of the research cycle, a new consensus needs to be built about how to use AI tools responsibly to benefit scientific research. APP is an effort to build such a consensus at the stage of initial publication. Similar protocols may emerge in other stages of the research cycle, and it would be interesting to study how they interact with each other.

5 Conclusion and Further Discussion

In conclusion, in this paper we proposed the Agentic Publication Protocol as a lightweight format for making scientific publications more organized, reproducible, informative, and interactive. Instead of static knowledge objects such as manuscripts and code, an APP publication is a “live” representation of a research work that interacts with readers. By organizing information in the paper in a more modular way and providing more explicit reproduction instructions, an APP publication aims to reduce the burden of understanding and reproducing a research work. In addition, APP provides a way for authors to share important know-how that is usually not included in traditional publication. The goal of APP is to make scientific publications more agentic while also preserving their traditional roles as records of scientific discovery and carriers of credit and priority.

In addition to proposing the APP protocol itself, this paper also provides two layers of agentic tools related to the protocol. The first layer consists of agent skills for publishing an APP paper, which guide authors through the workflow of preparing an APP publication. This layer also includes auxiliary skills that are useful for authors or readers. The second layer consists of agentic development tools, which help evaluate and improve the protocol and the skills in the protocol repository. We hope these tools can make it easier for researchers to adopt and contribute to the development of this new protocol. Our small-scale test shows that the APP agents have an advantage in grounding and honesty compared with a general agent with the same LLM.

In the final part of this paper, we zoom out to consider how the growing use of LLM agents may reshape scientific research more broadly. APP is an initial step toward a broader transformation of the research workflow. Scientific research can be viewed as a self-organized network in which each node has inputs, processing, and outputs:

- **Input:** gathering information from the literature, connecting related works, and importing prior methods.
- **Processing:** producing new research through analytic work, numerical simulations, experiments, or theory building.
- **Output:** sharing results through writing, posting, peer review, journal publication, lectures, discussion, video, audio, and other forms of interaction.

AI agents are bringing significant change to all three stages of research work. They can increase the degree of input by helping researchers search, connect, and operationalize prior work. They can accelerate processing by assisting with brainstorming, numerical simulations, analytic derivations, and even carrying out experiments. They can expand output by making publications interactive and usable as computational objects rather than static documents, which is the focus of APP in the current work. With AI agents, information transmission between research works becomes more efficient, informative, and interactive. The research network previously formed by human researchers is now gradually becoming a **self-organized human-AI network**. We expect such change to have the potential to drive a “phase transition” in the dynamics of the

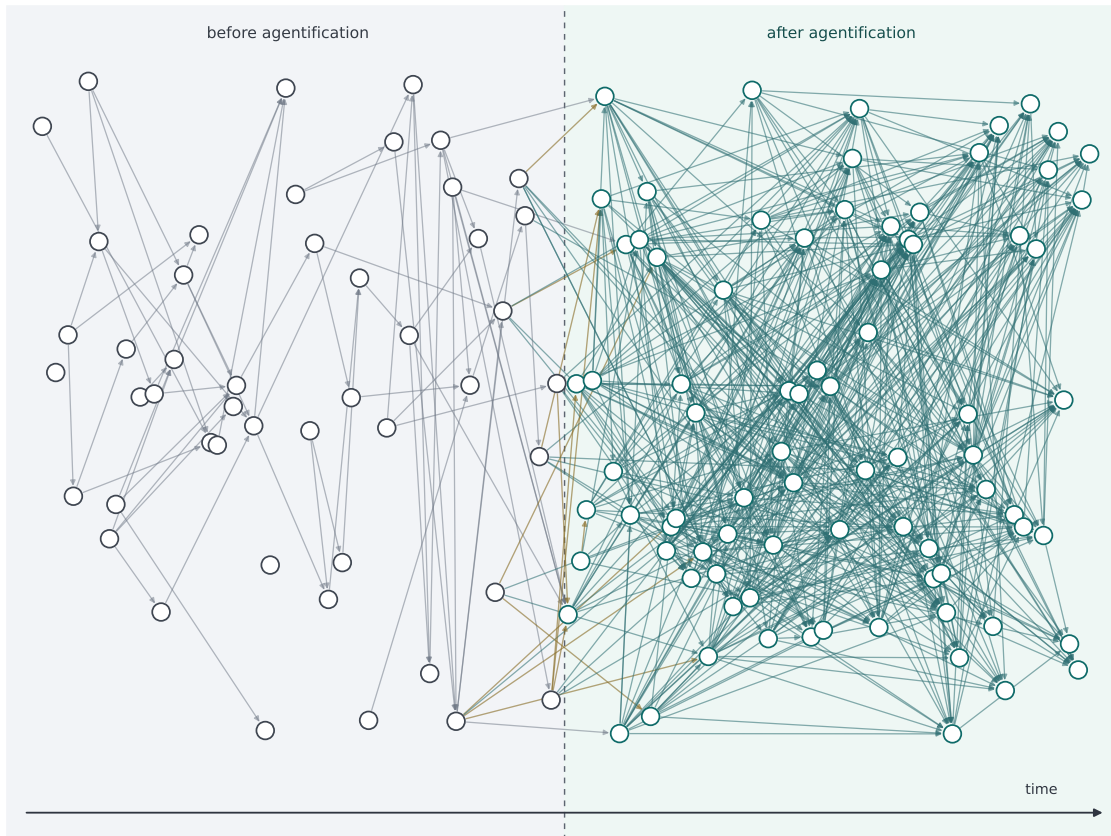


Figure 5: Schematic illustration of the qualitative change in the research network introduced by agentification. Each circular node represents a research work, and directed links point forward in time.

research network, where the pace of innovation is no longer limited by the communication and collaboration burden of individual researchers. This transition is illustrated in Fig. 5. It would be interesting to look for more precise and quantitative characterizations of such a phase transition and its implications. We leave this for future work.

Acknowledgments

We would like to thank Abhinava Chatterjee, Chaoxing Liu, and Quansheng Wu for testing the protocol and providing helpful feedback. In writing this paper, we used AI agents including Codex, Claude Code, TeXRA.ai for drafting part of the content, editing the language, and generating the figures. This paper is supported by the Simons Foundation (XLQ) and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC-2111 - 390814868 (SL).

Data and Code Availability

The data and code associated with this work include the [APP protocol repository](#), the [development repository](#) (which is also the APP paper repository for this work), and the example papers and `compare-app` benchmark data in [our Hugging Face dataset](#).

References

- [1] Monya Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604):452–454, 2016. doi: 10.1038/533452a.
- [2] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.
- [3] Victoria Stodden, Jennifer Seiler, and Zhaokun Ma. An empirical analysis of journal policy effectiveness for computational reproducibility. *Proceedings of the National Academy of Sciences*, 115(11):2584–2589, 2018. doi: 10.1073/pnas.1708290115.
- [4] Michael Polanyi. *The Tacit Dimension*. University of Chicago Press, Chicago, 2009. ISBN 9780226672984. First published 1966; reissued with a new foreword by Amartya Sen.
- [5] Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, Anima Anandkumar, Karianne Bergen, Carla P. Gomes, Shirley Ho, Pushmeet Kohli, Joan Lasenby, Jure Leskovec, Tie-Yan Liu, Arjun Manrai, Debora Marks, Bharath Ramsundar, Le Song, Jimeng Sun, Jian Tang, Petar Veličković, Max Welling, Linfeng Zhang, Connor W. Coley, Yoshua Bengio, and Marinka Zitnik. Scientific discovery in the age of artificial intelligence. *Nature*, 620(7972):47–60, 2023. doi: 10.1038/s41586-023-06221-2. URL <https://doi.org/10.1038/s41586-023-06221-2>.
- [6] Xiao-Liang Qi. The agentification of scientific research: A physicist’s perspective, 2026. URL <https://arxiv.org/abs/2604.14718>. arXiv:2604.14718.
- [7] Sébastien Bubeck, Christian Coester, Ronen Eldan, et al. Early science acceleration experiments with GPT-5, 2025. URL <https://arxiv.org/abs/2511.16072>. arXiv:2511.16072.
- [8] Sirui Lu, Zhijing Jin, Terry Jingchen Zhang, et al. Can theoretical physics research benefit from language agents?, 2025. URL <https://arxiv.org/abs/2506.06214>. arXiv:2506.06214.
- [9] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. doi: 10.1038/s41586-023-06792-0.
- [10] Andres M. Bran, Sam Cox, Oliver Schilter, et al. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. doi: 10.1038/s42256-024-00832-8. URL <https://doi.org/10.1038/s42256-024-00832-8>.
- [11] Jiaxuan Liu, Tiannian Zhu, Caiyuan Ye, et al. VASPilot: MCP-facilitated multi-agent intelligence for autonomous VASP simulations, 2025. URL <https://arxiv.org/abs/2508.07035>. arXiv:2508.07035.
- [12] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, Khaled Saab, Dan Popovici, Jacob Blum, Fan Zhang, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Pushmeet Kohli, Yossi Matias, Andrew Carroll, Kavita Kulkarni, Nenad Tomasev, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R. D. Costa, José R. Penadés, Gary Peltz, Yunhan Xu, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. Towards an AI co-scientist, 2025. URL <https://arxiv.org/abs/2502.18864>. arXiv:2502.18864.
- [13] Michael Y. Li, Emily B. Fox, and Noah D. Goodman. Automated statistical model discovery with language models, 2024. URL <https://arxiv.org/abs/2402.17879>. arXiv:2402.17879.
- [14] Michael P. Brenner, Vincent Cohen-Addad, and David Woodruff. Solving an open problem in theoretical physics using AI-assisted discovery, 2026. URL <https://arxiv.org/abs/2603.04735>. arXiv:2603.04735.
- [15] Alfredo Guevara, Alexandru Lupsasca, David Skinner, et al. Single-minus graviton tree amplitudes are nonzero, 2026. URL <https://cdn.openai.com/pdf/graviton.pdf>. OpenAI preprint PDF.

- [16] Tony Feng, Trieu H. Trinh, Garrett Bingham, et al. Towards autonomous mathematics research, 2026. URL <https://arxiv.org/abs/2602.10177>. arXiv:2602.10177.
- [17] Daniel Zheng, Ingrid von Glehn, Yori Zwols, et al. AI co-mathematician: Accelerating mathematicians with agentic AI, 2026. URL <https://arxiv.org/abs/2605.06651>. arXiv:2605.06651.
- [18] George Tsoukalas, Anton Kovsharov, Sergey Shirobokov, et al. Advancing mathematics research with AI-driven formal proof search, 2026. URL <https://arxiv.org/abs/2605.22763>. arXiv:2605.22763.
- [19] OpenAI. An OpenAI model has disproved a central conjecture in discrete geometry, May 2026. URL <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>. Research announcement, with links to proof and companion remarks.
- [20] Noga Alon, Thomas F. Bloom, W. T. Gowers, Daniel Litt, Will Sawin, Arul Shankar, Jacob Tsimerman, Victor Wang, and Melanie Matchett Wood. Remarks on the disproof of the unit distance conjecture, 2026. URL <https://cdn.openai.com/pdf/74c24085-19b0-4534-9c90-465b8e29ad73/unit-distance-remarks.pdf>. Companion remarks on the OpenAI unit-distance result.
- [21] Chris Lu, Cong Lu, Robert Tjarko Lange, et al. The AI scientist: Towards fully automated open-ended scientific discovery, 2024. URL <https://arxiv.org/abs/2408.06292>. arXiv:2408.06292.
- [22] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL <https://arxiv.org/abs/2504.08066>. arXiv:2504.08066.
- [23] Samuel Schmidgall, Yusheng Su, Ze Wang, et al. Agent laboratory: Using LLM agents as research assistants. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. URL <https://aclanthology.org/2025.findings-emnlp.320/>.
- [24] Yi Zhou. From paper to program: Accelerating quantum many-body algorithm development via a multi-stage LLM-assisted workflow, 2026. URL <https://arxiv.org/abs/2604.04089>. arXiv:2604.04089.
- [25] Ken Deng, Xiangfei Wang, Guijing Duan, et al. Towards verifiable and self-correcting AI physicists for quantum many-body simulations, 2026. URL <https://arxiv.org/abs/2604.00149>. arXiv:2604.00149.
- [26] Peter Cha, Paul Ginsparg, Felix Wu, Juan Carrasquilla, Peter L. McMahon, and Eun-Ah Kim. Attention-based quantum tomography. *Machine Learning: Science and Technology*, 3(1):01LT01, 2022. doi: 10.1088/2632-2153/ac362b. URL <https://arxiv.org/abs/2006.12469>. arXiv:2006.12469.
- [27] Ilektra Karakosta-Amarantidou, Raja Yehia, and Matteo Schiavon. Free-space model for a balloon-based quantum network, 2024. URL <https://arxiv.org/abs/2412.03356>. arXiv:2412.03356.
- [28] Michael A. Perlin, Zain H. Saleem, Martin Suchara, and James C. Osborn. Quantum circuit cutting with maximum likelihood tomography. *npj Quantum Information*, 7:64, 2021. doi: 10.1038/s41534-021-00390-6. URL <https://arxiv.org/abs/2005.12702>. arXiv:2005.12702.
- [29] Hatem Barghathi, Emanuel Casiano-Diaz, and Adrian Del Maestro. Particle partition entanglement of one dimensional spinless fermions. *Journal of Statistical Mechanics: Theory and Experiment*, 2017(8):083109, 2017. doi: 10.1088/1742-5468/aa819a. URL <https://arxiv.org/abs/1703.10587>. arXiv:1703.10587.
- [30] Adrián Pérez-Salinas, Patrick Emonts, Jordi Tura, and Vedran Dunjko. Multiple-basis representation of quantum states, 2024. URL <https://arxiv.org/abs/2411.03110>. arXiv:2411.03110.
- [31] Qiushi Liu, Zihao Hu, Haidong Yuan, and Yuxiang Yang. Optimal strategies of quantum metrology with a strict hierarchy. *Physical Review Letters*, 130:070803, 2023. doi: 10.1103/PhysRevLett.130.070803. URL <https://arxiv.org/abs/2203.09758>. arXiv:2203.09758.
- [32] Simanshu Kumar and Nandan S Bisht. OAM-induced lattice rotation reveals a fractional optimum in fault-tolerant GKP quantum sensing, 2026. URL <https://arxiv.org/abs/2605.13271>. arXiv:2605.13271.

- [33] Guillem Müller-Rigat, Anubhav Kumar Srivastava, Stanisław Kurdziałek, Grzegorz Rajchel-Mieldzioć, Maciej Lewenstein, and Irénée Frérot. Certifying the quantum fisher information from a given set of mean values: a semidefinite programming approach. *Quantum*, 7:1152, 2023. doi: 10.22331/q-2023-10-24-1152. URL <https://arxiv.org/abs/2306.12711>. arXiv:2306.12711.
- [34] Chirag Wadhwa and Mina Doosti. Learning quantum processes with quantum statistical queries. *Quantum*, 9:1739, 2025. doi: 10.22331/q-2025-05-12-1739. URL <https://arxiv.org/abs/2310.02075>. arXiv:2310.02075.
- [35] Boris Sokolov, Matteo A. C. Rossi, Guillermo García-Pérez, and Sabrina Maniscalco. Emergent entanglement structures and self-similarity in quantum spin chains. *Philosophical Transactions of the Royal Society A*, 380(2227):20200421, 2022. doi: 10.1098/rsta.2020.0421. URL <https://arxiv.org/abs/2007.06989>. arXiv:2007.06989.
- [36] Daniel Malz and Adam Smith. Topological two-dimensional floquet lattice on a single superconducting qubit. *Physical Review Letters*, 126:163602, 2021. doi: 10.1103/PhysRevLett.126.163602. URL <https://arxiv.org/abs/2012.01459>. arXiv:2012.01459.
- [37] Sean Bechhofer, Iain Buchan, David De Roure, et al. Why linked data is not enough for scientists. *Future Generation Computer Systems*, 29(2):599–611, 2013. doi: 10.1016/j.future.2011.08.004.
- [38] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.
- [39] Project Jupyter, Matthias Bussonnier, Jessica Forde, Jeremy Freeman, Brian Granger, Tim Head, Chris Holdgraf, Kyle Kelley, Gladys Nalvarte, Andrew Osheroff, M Pacer, Yuvi Panda, Fernando Pérez, Benjamin Ragan-Kelley, and Carol Willing. Binder 2.0 – reproducible, interactive, sharable environments for science at scale. In *Proceedings of the 17th Python in Science Conference*, pages 113–120, 2018. doi: 10.25080/Majora-4a1f417-011.
- [40] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL <http://jmlr.org/papers/v22/20-303.html>.
- [41] Association for Computing Machinery. Artifact review and badging – current (version 1.1). ACM publications policy, August 2020. URL <https://www.acm.org/publications/policies/artifact-review-and-badging-current>. Accessed 2026-06-10.
- [42] Hugging Face. Hugging Face Papers. Website, 2026. URL <https://huggingface.co/papers>. Hosts the paper-code index formerly maintained by Papers with Code; <https://huggingface.co/papers>; accessed 2026-06-10.
- [43] Jiachen Liu, Jiaxin Pei, Jintao Huang, Chenglei Si, Ao Qu, Xiangru Tang, Runyu Lu, Lichang Chen, Xiaoyan Bai, Haizhong Zheng, Carl Chen, Zhiyang Chen, Haojie Ye, Yujuan Fu, Zexue He, Zijian Jin, Zhenyu Zhang, Shangquan Sun, Maestro Harmon, John Dianzhuo Wang, Jianqiao Zeng, Jiachen Sun, Mingyuan Wu, Baoyu Zhou, Yuchen You, Shijian Lu, Yiming Qiu, Fan Lai, Yuan Yuan, Yao Li, Junyuan Hong, Ruihao Zhu, Beidi Chen, Alex Pentland, Ang Chen, Mosharaf Chowdhury, and Zechen Zhang. The last human-written paper: Agent-native research artifacts, 2026. URL <https://arxiv.org/abs/2604.24658>. arXiv:2604.24658.
- [44] Jiacheng Miao, Joe R. Davis, Yaohui Zhang, Jonathan K. Pritchard, and James Zou. Paper2Agent: Reimagining research papers as interactive and reliable AI agents, 2025. URL <https://arxiv.org/abs/2509.06917>. arXiv:2509.06917.
- [45] Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, November 2024. Open standard for connecting AI assistants to data sources and tools; accessed 2026-06-10.

- [46] Minju Seo, Jinheon Baek, Seongyun Lee, and Sung Ju Hwang. Paper2Code: Automating code generation from scientific papers in machine learning. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=3DcaUTjdKc>. arXiv:2504.17192.
- [47] Federico Bianchi, Owen Queen, Nitya Thakkar, Eric Sun, James Zou, et al. Exploring the use of AI authors and reviewers at Agents4Science. *Nature Biotechnology*, 44:11–14, 2026. doi: 10.1038/s41587-025-02963-8. URL <https://doi.org/10.1038/s41587-025-02963-8>.
- [48] Weixin Liang, Yuhui Zhang, Hancheng Cao, Binglu Wang, Daisy Yi Ding, Xinyu Yang, Kailas Vodrahalli, Siyu He, Daniel Scott Smith, Yian Yin, Daniel A. McFarland, and James Zou. Can large language models provide useful feedback on research papers? A large-scale empirical analysis. *NEJM AI*, 1(8): AIoa2400196, 2024. doi: 10.1056/AIoa2400196.
- [49] Giuseppe Russo Latona, Manoel Horta Ribeiro, Tim R. Davidson, Veniamin Veselovsky, and Robert West. The AI review lottery: Widespread AI-assisted peer reviews boost paper scores and acceptance rates, 2024. URL <https://arxiv.org/abs/2405.02150>. arXiv:2405.02150.
- [50] Weixin Liang, Yaohui Zhang, Zhengxuan Wu, et al. Quantifying large language model usage in scientific papers. *Nature Human Behaviour*, 9:2599–2609, 2025. doi: 10.1038/s41562-025-02273-8. URL <https://doi.org/10.1038/s41562-025-02273-8>.
- [51] Keigo Kusumegi, Xinyu Yang, Paul Ginsparg, et al. Scientific production in the era of large language models. *Science*, 390(6779):1240–1243, 2025. doi: 10.1126/science.adw3000. URL <https://doi.org/10.1126/science.adw3000>.
- [52] Riccardo Bertolo and Alessandro Antonelli. Generative AI in scientific publishing: Disruptive or destructive? *Nature Reviews Urology*, 21:1–2, 2024. doi: 10.1038/s41585-023-00836-w. URL <https://doi.org/10.1038/s41585-023-00836-w>.
- [53] ICML. ICML 2023 clarification on large language model policy. <https://icml.cc/Conferences/2023/llm-policy>, 2023. Accessed 2026-06-10.
- [54] Editorial. Tools such as ChatGPT threaten transparent science; here are our ground rules for their use. *Nature*, 613:612, 2023. doi: 10.1038/d41586-023-00191-1. Editorial.
- [55] H. Holden Thorp. ChatGPT is fun, but not an author. *Science*, 379(6630):313, 2023. doi: 10.1126/science.adg7879.
- [56] NeurIPS. NeurIPS 2025 policy on the use of large language models. <https://neurips.cc/Conferences/2025/LLM>, 2025. Accessed 2026-06-10.
- [57] ICLR 2026 Program Chairs. Policies on large language model usage at ICLR 2026. <https://blog.iclr.cc/2025/08/26/policies-on-large-language-model-usage-at-iclr-2026/>, August 2025. Accessed 2026-06-10.